

Stream Pipe

Difficulty: Hard

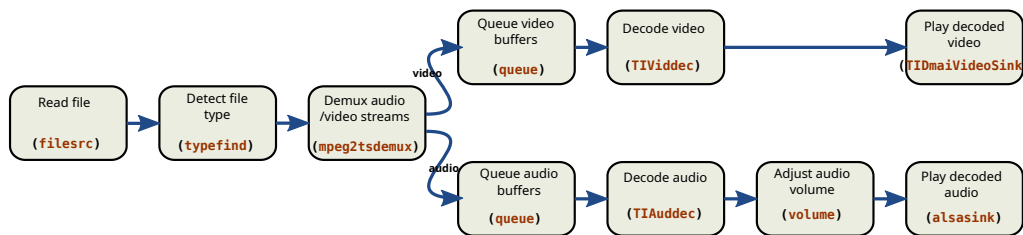
Expect Time: 60-90min

Author: Chen, YanHau

Streaming services are a very common application in modern society. Referring to the architecture diagrams of systems such as GStreamer or FFmpeg, they typically involve a series of operations such as demuxing, decoding, encoding, and muxing, which package raw data into media containers.

Example GStreamer Pipeline

The goals of GStreamer are to separate the application (e.g. Video player, Video editor, etc.) from the streaming media complexity (e.g. hardware acceleration, remoteness)
GStreamer - streaming media
D-Bus - inter process communication
Use **gst-launch** command to create the GStreamer pipeline
gst-rtsp-server - v4l2src ! video/x-raw,width=1280,height=720 ! omxh264enc ! video/x-h264,profile=baseline ! h264parse config-interval=1 ! rtph264pay name=pay0 pt=96



```
gst-launch filesrc location="video.ts" ! typefind ! mpeg2tsdemux name=demux \
    demux. ! 'video/x-h264' ! queue ! TIViddec ! TIDmaVideoSink \
    demux. ! 'audio/mpeg' ! queue ! TIAuddec ! volume volume=5 ! alsasink
```

In this problem, we will implement a pipeline that simulates a media processing workflow. The following are several core components:

- Buffer: The type used to store the actual data, defined as an alias of `std::array<uint8_t, 10>`.
- RawChunk: The basic input type defined by the problem setter; all transformations begin from this type.
- *Frame and *Packet: Struct includes `{ Buffer data }` member.

Codecs & Muldex

In `media.h`, six major types are defined: `H264Frame`, `H265Frame`, `H266Frame`, `VP8Frame`, `VP9Frame`, and `AV1Frame`, corresponding to the `codecs::*enc` and `codecs::*dec` series of functions. Similarly, muxers and demuxers implement a series of `<Frame>To<Packet>` or `<Packet>To<Frame>` functions. These methods that attempt to convert A into B are referred to as *pipeable functions*. A *pipeable function* returns an instance of the class `Pipe<FROM, TO>`.

```
// For example, `h264enc` returns an instance of `Pipe<RawChunk, H264Frame>`.
// h264enc is a `pipeable function`
inline Pipe<RawChunk, H264Frame> h264enc() {
    return Pipe<RawChunk, H264Frame>([](RawChunk in, H264Frame& out) {
        process(in.data, out.data, 0x65);
        return true;
    });
}
```

Pipe

For class `Pipe<FROM, TO>`, you need to implement two key methods:

- `Pipe::pipe`, which allows registering a function of the form `bool(FROM in, TO& out)`.
- `bool Pipe::process(FROM in, TO& out) const`, which attempts to invoke the pipeable function defined in `media_type.h`. It returns `true` if the conversion succeeds; otherwise, it returns `false`.

You also need to overload `operator|(PipeA, PipeB)`. It takes two Pipe instances, and if they can be connected, it attempts to pass data through the combined pipeline for conversion.

```
template <typename SRC, typename MID, typename DST>
auto operator|(Pipe<SRC, MID> src, Pipe<MID, DST> sink) → Pipe<SRC, DST> { }
```

In `case*.h`, a set of pipelines is define:

```
auto pipeline = codecs::h264enc()
    | muxdex::h264ToMP4()
    | muxdex::MP4Toh264();

RawChunk chunk;
H264Frame frame;
pipeline.process(chunk, frame);
```

The data is expected to flow through the pipeline:

```
Pipe<RawChunk, H264Frame> | Pipe<H264Frame, MP4Packet> | Pipe<MP4Packet,
H264Frame> .
```

Since `operator|` is used to chain multiple pipes, the final result should be `Pipe<RawChunk, H264Frame>`. Therefore, `pipeline.process` takes `RawChunk` as input and produces `H264Frame` as output.

Complete the missing implementation in `media_pipe.h` so that all test cases can run successfully.

Input

1. Complete the required class implementation. The following files already contain some implementations:
 - `media.h`
 - `media_pipe.h`
2. The Online Judge system will replace the following files. **DON'T** write implementation in those file:
 - `entrypoint.cpp`
 - `test.h`
 - `case.h`
3. The test cases for student available in `case*.h` .

Output

Online Judge will run the test:

1. **DON' T** output any information to the terminal.
2. When any test failed, Online Judge will try to list error messages.